

Git — Complete Commands Reference

Basics se Mastery tak • Daily Use • Advanced Workflows • Senior Engineer Cheatsheet

basic

beginner commands

mid

intermediate

adv

advanced / power user

danger!

destructive — dhyan se

1. Setup aur Configuration — Pehli Baar

Command	Level	Kya karta hai
<code>git --version</code>	basic	Git version check karna
<code>git config --global user.name 'Alice'</code>	basic	apna naam set karna (sabhi repos ke liye)
<code>git config --global user.email 'a@b.com'</code>	basic	email set karna
<code>git config --global core.editor vim</code>	basic	default editor set karna (vim/nano/code)
<code>git config --global init.defaultBranch main</code>	basic	default branch naam 'main' rakhna (master ki jagah)
<code>git config --global color.ui auto</code>	basic	colored output enable karna
<code>git config --global pull.rebase false</code>	basic	pull pe merge strategy set karna
<code>git config --global core.autocrlf input</code>	basic	line endings normalize karna (Linux/Mac)
<code>git config --list</code>	basic	sabhi config settings dekhna
<code>git config --list --show-origin</code>	basic	config setting kahan se aa rahi hai dekhna
<code>git config user.name</code>	basic	ek specific config value dekhna
<code>git config --global alias.st status</code>	basic	shortcut alias banana — ab 'git st' chalega
<code>git config --global alias.lg 'log --oneline --graph --all'</code>	basic	complex command ka alias banana
<code>git config --global --edit</code>	basic	global config file directly edit karna
<code>cat ~/.gitconfig</code>	basic	global config file directly dekhna

2. Repository — Banana aur Clone Karna

Command	Level	Kya karta hai
<code>git init</code>	basic	current directory mein naya git repo banana
<code>git init myproject</code>	basic	naya folder banana aur usme repo initialize karna
<code>git init --bare myrepo.git</code>	mid	bare repo banana (server pe hosting ke liye, no working tree)
<code>git clone https://github.com/user/repo.git</code>	basic	remote repo clone karna
<code>git clone https://repo.git mydir</code>	basic	specific folder naam se clone karna
<code>git clone --depth=1 https://repo.git</code>	mid	sirf latest commit clone karna (fast, shallow clone)
<code>git clone --branch develop https://repo.git</code>	mid	specific branch clone karna

Command	Level	Kya karta hai
<code>git clone --single-branch -b main https://repo.git</code>	mid	sirf ek branch clone karna (disk save)
<code>git clone git@github.com:user/repo.git</code>	basic	SSH se clone karna (password-less)
<code>git remote -v</code>	basic	configured remote URLs dekhna
<code>git remote add origin https://repo.git</code>	basic	existing local repo mein remote add karna
<code>git remote set-url origin https://new-url.git</code>	mid	remote URL change karna
<code>git remote rename origin upstream</code>	mid	remote ka naam badalna
<code>git remote remove origin</code>	mid	remote connection hatana

3. Basic Workflow — Roz Ka Kaam

Git ka basic cycle: Working Directory → Staging Area (Index) → Local Repo → Remote Repo

Command	Level	Kya karta hai
<code>git status</code>	basic	working directory aur staging area ki current state dekhna
<code>git status -s</code>	basic	short format mein status (M=modified, A=added, ??=untracked)
<code>git add file.txt</code>	basic	file ko staging area mein add karna
<code>git add .</code>	basic	sabhi changes stage karna (current directory)
<code>git add -A</code>	basic	sabhi changes stage karna (deleted files bhi — root se)
<code>git add -p</code>	mid	interactively chunks select karke stage karna (hunk by hunk)
<code>git add -u</code>	mid	sirf tracked files ke changes stage karna (new files nahi)
<code>git add src/</code>	basic	specific directory ke sabhi changes stage karna
<code>git add *.js</code>	basic	wildcard se files stage karna
<code>git commit -m 'feat: add login page'</code>	basic	staged changes commit karna with message
<code>git commit -am 'fix: bug fixed'</code>	basic	tracked files stage + commit ek step mein (new files nahi)
<code>git commit --amend -m 'new message'</code>	mid	last commit message badalna (push se pehle!)
<code>git commit --amend --no-edit</code>	mid	last commit mein changes add karna, message same rakhna
<code>git commit --allow-empty -m 'trigger CI'</code>	mid	empty commit banana (CI/CD trigger ke liye)
<code>git diff</code>	basic	unstaged changes dekhna (working dir vs staging)
<code>git diff --staged</code>	basic	staged changes dekhna (staging vs last commit)
<code>git diff HEAD</code>	basic	sabhi changes dekhna (working dir vs last commit)
<code>git diff main..feature</code>	mid	do branches ke beech diff dekhna
<code>git diff --stat</code>	basic	diff summary — kitni files, lines changed

Command	Level	Kya karta hai
<code>git diff --word-diff</code>	mid	word level diff (line level nahi)

4. Branches — Banana, Switch, Delete

Command	Level	Kya karta hai
<code>git branch</code>	basic	local branches list karna (current branch * se mark)
<code>git branch -a</code>	basic	local + remote sabhi branches dekhna
<code>git branch -r</code>	basic	sirf remote branches dekhna
<code>git branch -v</code>	basic	branches + last commit message dekhna
<code>git branch --merged</code>	mid	main mein merge ho chuki branches dekhna
<code>git branch --no-merged</code>	mid	abhi merge nahi hui branches dekhna
<code>git branch feature/login</code>	basic	naya branch banana (switch nahi karta)
<code>git checkout -b feature/login</code>	basic	naya branch banana aur switch karna
<code>git switch -c feature/login</code>	basic	naya branch banana aur switch karna (modern syntax)
<code>git switch main</code>	basic	existing branch pe switch karna (modern)
<code>git checkout main</code>	basic	branch switch karna (purana syntax)
<code>git checkout -</code>	basic	pichli branch pe wapas jaana
<code>git branch -m old-name new-name</code>	mid	branch rename karna
<code>git branch -m new-name</code>	mid	current branch rename karna
<code>git branch -d feature/login</code>	basic	merged branch delete karna (safe)
<code>git branch -D feature/login</code>	danger!	force delete — merge na hui ho tab bhi
<code>git push origin --delete feature/login</code>	mid	remote branch delete karna
<code>git branch feature/login origin/feature/login</code>	mid	remote branch ko track karna locally
<code>git checkout --track origin/feature</code>	mid	remote branch track karte hue local banana
<code>git push -u origin feature/login</code>	mid	branch push karo aur upstream set karo

5. Merge — Branches Jodna

Command	Level	Kya karta hai
<code>git merge feature/login</code>	basic	feature branch ko current branch mein merge karna
<code>git merge --no-ff feature/login</code>	mid	fast-forward ke bawajood merge commit banana (history clear)
<code>git merge --squash feature/login</code>	mid	saari commits ek mein squash karke merge karna

Command	Level	Kya karta hai
<code>git merge --abort</code>	basic	conflict ke beech mein merge cancel karna
<code>git merge --continue</code>	basic	conflict resolve karne ke baad merge continue karna
<code>git mergetool</code>	mid	visual merge tool open karna conflicts ke liye
<code>git diff --merge-base main feature</code>	adv	merge base se diff dekhna
<code>git log --merges</code>	mid	sirf merge commits dekhna
<code>git log --no-merges</code>	mid	merge commits exclude karke log dekhna

Conflict resolve karne ka flow:

1) `git status` → 2) conflicted files edit karo → 3) `git add file` → 4) `git commit`

6. Rebase — Clean History Banana

Rebase = commits ko ek nayi base pe replay karna. Merge se cleaner history milti hai lekin shared branches pe kabhi mat karo!

Command	Level	Kya karta hai
<code>git rebase main</code>	mid	current branch ko main ke latest commit pe rebase karna
<code>git rebase -i HEAD~3</code>	adv	last 3 commits interactive rebase — squash, edit, reorder, drop
<code>git rebase -i HEAD~3 --autosquash</code>	adv	fixup! prefix wale commits auto-squash karna
<code>git rebase --onto main feature bugfix</code>	adv	bugfix ko feature ki jagah main pe rebase karna
<code>git rebase --abort</code>	basic	rebase cancel karna, pehli state pe wapas
<code>git rebase --continue</code>	basic	conflict resolve karne ke baad rebase continue karna
<code>git rebase --skip</code>	adv	current commit skip karna rebase mein
<code>git pull --rebase</code>	mid	pull karo rebase se (merge commit avoid karna)
<code>git pull --rebase=interactive</code>	adv	pull pe interactive rebase

Interactive rebase commands: `pick`=rakhna `reword`=message badalna `edit`=pause `squash`=pichle mein milao `fixup`=squash+message hatao `drop`=delete

7. Remote — Push, Pull, Fetch

Command	Level	Kya karta hai
<code>git fetch origin</code>	basic	remote changes download karo (merge/rebase nahi karta)
<code>git fetch --all</code>	basic	sabhi remotes se fetch karna
<code>git fetch --prune</code>	mid	deleted remote branches ki local references clean karna
<code>git pull</code>	basic	fetch + merge (current branch ke liye)
<code>git pull origin main</code>	basic	origin ke main se pull karna
<code>git pull --rebase origin main</code>	mid	pull with rebase — cleaner history

Command	Level	Kya karta hai
<code>git pull --ff-only</code>	mid	sirf fast-forward possible ho tabhi pull karna
<code>git push origin main</code>	basic	main branch push karna
<code>git push -u origin feature/login</code>	basic	branch push karo aur upstream track set karo
<code>git push --all origin</code>	mid	sabhi local branches push karna
<code>git push origin --tags</code>	basic	sabhi tags push karna
<code>git push origin v1.0.0</code>	basic	specific tag push karna
<code>git push --force-with-lease</code>	adv	safe force push — doosre ka kaam overwrite nahi karega
<code>git push --force</code>	danger!	force push — remote history overwrite karta hai
<code>git push origin :feature/old</code>	mid	remote branch delete karna (colon syntax)
<code>git ls-remote origin</code>	adv	remote ke sabhi refs dekhna bina fetch kiye
<code>git remote show origin</code>	mid	remote ki detailed info — branches, tracking
<code>git branch -vv</code>	mid	local branches aur unka remote tracking status

8. Log aur History — Commits Dekhna

Command	Level	Kya karta hai
<code>git log</code>	basic	commit history dekhna (newest first)
<code>git log --oneline</code>	basic	ek line per commit — short hash + message
<code>git log --oneline --graph --all</code>	basic	sabhi branches ka tree graph dekhna
<code>git log --oneline -10</code>	basic	last 10 commits
<code>git log --author='Alice'</code>	basic	specific author ke commits
<code>git log --since='2 weeks ago'</code>	basic	last 2 hafte ke commits
<code>git log --since='2024-01-01' --until='2024-06-01'</code>	mid	date range ke commits
<code>git log --grep='fix'</code>	basic	commit message mein 'fix' wale commits
<code>git log -S 'function_name'</code>	adv	specific string add/remove karne wale commits (pickaxe)
<code>git log -G 'regex_pattern'</code>	adv	regex pattern change karne wale commits
<code>git log -- file.txt</code>	basic	specific file ke commits
<code>git log --follow file.txt</code>	mid	file ke renames ke paar bhi history track karna
<code>git log main..feature</code>	mid	feature mein hai lekin main mein nahi wale commits
<code>git log --stat</code>	basic	har commit mein changed files ka summary
<code>git log --patch</code>	mid	har commit ka full diff bhi dikhana
<code>git log --format='%h %an %s'</code>	mid	custom format — hash, author, subject

Command	Level	Kya karta hai
<code>git log --format='%H' head -5</code>	mid	last 5 commits ke full hashes
<code>git shortlog -sn</code>	mid	author ke hisaab se commit count (leaderboard)
<code>git show abc1234</code>	basic	specific commit ka detail + diff dekhna
<code>git show abc1234:file.txt</code>	mid	specific commit pe file ka content dekhna
<code>git show HEAD</code>	basic	last commit dekhna
<code>git show HEAD~2</code>	mid	2 commits pehle wala dekhna

9. Stash — Kaam Temporarily Save Karna

Stash = uncommitted changes temporarily side mein rakhna — branch switch karne se pehle useful

Command	Level	Kya karta hai
<code>git stash</code>	basic	current uncommitted changes stash karna
<code>git stash push -m 'WIP: login page'</code>	mid	meaningful naam ke saath stash karna
<code>git stash -u</code>	basic	untracked files bhi stash karna
<code>git stash -a</code>	adv	ignored files bhi stash karna
<code>git stash list</code>	basic	sabhi stashes dekhna
<code>git stash show</code>	basic	latest stash ka summary dekhna
<code>git stash show -p</code>	mid	latest stash ka full diff dekhna
<code>git stash show stash@{2}</code>	mid	specific stash ka summary
<code>git stash pop</code>	basic	latest stash wapas lao aur stash list se hatao
<code>git stash apply</code>	mid	stash wapas lao lekin stash list mein rakho
<code>git stash apply stash@{2}</code>	mid	specific stash apply karna
<code>git stash drop stash@{1}</code>	mid	specific stash delete karna
<code>git stash clear</code>	danger!	sabhi stashes delete karna
<code>git stash branch feature/new stash@{0}</code>	adv	stash se naya branch banana
<code>git stash pop --index</code>	adv	staged state bhi restore karna

10. Tags — Releases Mark Karna

Command	Level	Kya karta hai
<code>git tag</code>	basic	sabhi tags list karna
<code>git tag -l 'v1.*'</code>	basic	pattern se tags filter karna
<code>git tag v1.0.0</code>	basic	lightweight tag banana (sirf pointer)

Command	Level	Kya karta hai
<code>git tag -a v1.0.0 -m 'Release 1.0'</code>	mid	annotated tag banana (message, author, date included)
<code>git tag -a v1.0.0 abc1234</code>	mid	specific commit pe tag lagana
<code>git show v1.0.0</code>	basic	tag ki details dekhna
<code>git push origin v1.0.0</code>	basic	specific tag push karna
<code>git push origin --tags</code>	basic	sabhi tags push karna
<code>git push origin :refs/tags/v1.0.0</code>	mid	remote tag delete karna
<code>git tag -d v1.0.0</code>	basic	local tag delete karna
<code>git checkout v1.0.0</code>	mid	specific tag pe checkout karna (detached HEAD)
<code>git describe --tags</code>	mid	nearest tag + commits since + hash
<code>git describe --tags --abbrev=0</code>	mid	nearest tag ka naam dekhna

11. Undo aur Reset — Galti Theek Karna

WARNING: reset --hard aur clean -fd permanent hain — pehle backup lo ya stash karo!

Command	Level	Kya karta hai
<code>git restore file.txt</code>	mid	unstaged changes discard karna (working dir reset)
<code>git restore --staged file.txt</code>	mid	file unstage karna (staging se hatao)
<code>git restore --source=HEAD~2 file.txt</code>	adv	file ko 2 commits pehle wali state mein restore karna
<code>git checkout -- file.txt</code>	basic	file discard karna (purana syntax — restore prefer karo)
<code>git reset HEAD file.txt</code>	basic	file unstage karna (purani syntax)
<code>git revert abc1234</code>	mid	commit ka reverse commit banana — history safe rehti hai
<code>git revert HEAD</code>	basic	last commit revert karna
<code>git revert HEAD~3..HEAD</code>	adv	last 3 commits revert karna
<code>git revert --no-commit HEAD~3..HEAD</code>	adv	revert karo lekin commit mat karo abhi
<code>git reset --soft HEAD~1</code>	mid	last commit undo — changes staged mein rahenge
<code>git reset --mixed HEAD~1</code>	mid	last commit undo — changes unstaged honge (default)
<code>git reset --hard HEAD~1</code>	danger!	last commit undo — changes bhi delete!
<code>git reset --hard origin/main</code>	danger!	local branch ko remote ke saath sync karna (local changes lost!)
<code>git clean -fd</code>	danger!	untracked files aur directories delete karna
<code>git clean -fdn</code>	mid	dry-run — kya delete hoga preview karo (safe)

Command	Level	Kya karta hai
<code>git clean -fdx</code>	danger!	ignored files bhi delete karna
<code>git commit --amend</code>	mid	last commit modify karna (push se pehle only!)

12. Cherry-pick — Specific Commits Lena

Command	Level	Kya karta hai
<code>git cherry-pick abc1234</code>	mid	ek specific commit current branch pe apply karna
<code>git cherry-pick abc1234 def5678</code>	mid	multiple commits cherry-pick karna
<code>git cherry-pick abc1234..def5678</code>	adv	commit range cherry-pick karna (abc exclusive)
<code>git cherry-pick abc1234^..def5678</code>	adv	commit range (abc inclusive)
<code>git cherry-pick -n abc1234</code>	adv	cherry-pick karo lekin auto-commit mat karo
<code>git cherry-pick --abort</code>	basic	cherry-pick cancel karna conflict ke beech
<code>git cherry-pick --continue</code>	basic	conflict resolve karne ke baad continue
<code>git cherry-pick -x abc1234</code>	adv	original commit hash message mein add karna (traceability)

13. Reflog — Khoya Hua Kaam Wapas Lana

Reflog Git ka 'safety net' hai — har HEAD movement record karta hai. Accidentally deleted branch bhi wapas aa sakti hai!

Command	Level	Kya karta hai
<code>git reflog</code>	mid	sabhi HEAD movements ki history dekhna
<code>git reflog show branch-name</code>	mid	specific branch ka reflog dekhna
<code>git reflog --relative-date</code>	mid	timestamps ke saath reflog
<code>git checkout HEAD@{3}</code>	adv	3 moves pehle ki state mein jaana
<code>git reset --hard HEAD@{2}</code>	danger!	2 moves pehle ki state pe hard reset karna
<code>git branch recovered HEAD@{5}</code>	adv	accidentally deleted branch recover karna
<code>git reflog expire --expire=90.days.ago --all</code>	adv	90 din se purani reflog entries clean karna

14. Blame aur Bisect — Bug Dhundna

Command	Level	Kya karta hai
<code>git blame file.txt</code>	mid	har line kab aur kisne last modify ki
<code>git blame -L 10,20 file.txt</code>	mid	sirf lines 10-20 ka blame
<code>git blame -w file.txt</code>	mid	whitespace changes ignore karna blame mein

Command	Level	Kya karta hai
<code>git blame --follow -L 5,10 file.txt</code>	adv	renames ke paar bhi blame track karna
<code>git log -p -S 'buggy_function'</code>	mid	specific function change karne wala commit dhundna
<code>git bisect start</code>	adv	binary search start karna bug-introducing commit dhundne ke liye
<code>git bisect bad</code>	adv	current commit mein bug hai — mark karna
<code>git bisect good v1.0.0</code>	adv	yeh commit theek tha — mark karna
<code>git bisect good / git bisect bad</code>	adv	Git checkout karta hai — test karo aur mark karo
<code>git bisect reset</code>	adv	bisect khatam — original branch pe wapas
<code>git bisect run ./test.sh</code>	adv	automatic bisect — script se good/bad determine karna
<code>git bisect log</code>	adv	bisect session ka log dekhna

15. Worktree — Ek Saath Multiple Branches

Worktree = same repo ko alag directories mein checkout karna — stash/switch kiye bina do branches pe kaam karo

Command	Level	Kya karta hai
<code>git worktree add ../hotfix hotfix-branch</code>	adv	hotfix branch ko alag directory mein checkout karna
<code>git worktree add -b new-branch ../dir</code>	adv	naya branch banana aur worktree add karna
<code>git worktree list</code>	adv	sabhi worktrees dekhna
<code>git worktree remove ../hotfix</code>	adv	worktree remove karna
<code>git worktree prune</code>	adv	stale worktree references clean karna

16. Submodules — Repo Ke Andar Repo

Command	Level	Kya karta hai
<code>git submodule add https://repo.git lib/</code>	adv	submodule add karna
<code>git submodule init</code>	adv	submodule initialize karna (.gitmodules se)
<code>git submodule update</code>	adv	submodules latest state pe update karna
<code>git submodule update --init --recursive</code>	adv	sabhi nested submodules initialize + update karna
<code>git clone --recurse-submodules https://repo.git</code>	adv	submodules ke saath clone karna
<code>git submodule foreach git pull origin main</code>	adv	sabhi submodules mein command run karna
<code>git submodule status</code>	adv	submodules ki current state dekhna
<code>git submodule deinit lib/mymodule</code>	adv	submodule deregister karna
<code>git rm lib/mymodule</code>	adv	submodule remove karna

17. Git Hooks — Automation

Hooks = `.git/hooks/` folder mein scripts — git events pe automatically chalte hain

Command	Level	Kya karta hai
<code>ls .git/hooks/</code>	basic	available hook samples dekhna
<code>cat .git/hooks/pre-commit.sample</code>	basic	pre-commit hook sample dekhna
<code>cp .git/hooks/pre-commit.sample .git/hooks/pre-commit</code>	adv	hook enable karna (sample ko active banana)
<code>chmod +x .git/hooks/pre-commit</code>	adv	hook executable banana
<code>git commit --no-verify</code>	adv	hooks skip karke commit karna
<code>git push --no-verify</code>	adv	pre-push hook skip karna

Common hooks: `pre-commit` (lint/test), `commit-msg` (message format), `pre-push` (tests), `post-merge` (dependencies install)

18. Search — Code aur History Mein Dhundna

Command	Level	Kya karta hai
<code>git grep 'function login'</code>	mid	working directory mein string dhundna (fast)
<code>git grep -n 'TODO'</code>	mid	line numbers ke saath grep
<code>git grep -l 'password'</code>	mid	sirf file names dikhana jisme match mila
<code>git grep -i 'error'</code>	mid	case-insensitive grep
<code>git grep 'pattern' -- '*.py'</code>	mid	sirf Python files mein grep karna
<code>git grep 'pattern' HEAD~5</code>	adv	5 commits pehle ke code mein grep karna
<code>git log -S 'deleted_function'</code>	adv	yeh string add/remove karne wala commit dhundna
<code>git log --all --full-history -- '**/file.txt'</code>	adv	deleted file ka history dhundna
<code>git log --diff-filter=D -- file.txt</code>	adv	file delete karne wala commit dhundna

19. Maintenance aur Optimization

Command	Level	Kya karta hai
<code>git gc</code>	adv	garbage collection — loose objects pack karna
<code>git gc --aggressive</code>	adv	aggressive optimization (slow but thorough)
<code>git prune</code>	adv	unreachable objects delete karna
<code>git fsck</code>	adv	repository integrity check karna
<code>git fsck --lost-found</code>	adv	dangling commits/blobs dhundna
<code>git count-objects -vH</code>	adv	repo mein objects ka count aur size
<code>git remote prune origin</code>	mid	deleted remote branches ki stale refs clean karna

Command	Level	Kya karta hai
<code>git fetch --prune</code>	mid	fetch + prune ek saath
<code>git repack -ad</code>	adv	objects repack karna (repo size kam karna)
<code>git replace --graft parent child</code>	adv	commit history rewrite karna (grafts)
<code>git filter-branch --tree-filter 'rm -f password.txt' HEAD</code>	danger!	file ko poori history se delete karna (BFG prefer karo)
<code>bfg --delete-files password.txt</code>	adv	BFG Repo Cleaner — sensitive files history se hatana (faster)

20. .gitignore aur .gitattributes

Command	Level	Kya karta hai
<code>cat .gitignore</code>	basic	.gitignore file dekhna
<code>echo 'node_modules/' >> .gitignore</code>	basic	pattern ignore list mein add karna
<code>git check-ignore -v file.txt</code>	mid	file ignore ho rahi hai ya nahi aur kyun
<code>git status --ignored</code>	mid	ignored files bhi status mein dikhana
<code>git rm --cached file.txt</code>	mid	file ko git tracking se hatao lekin disk pe rakho
<code>git rm --cached -r node_modules/</code>	mid	directory ko tracking se hatana
<code>git add -f file.txt</code>	mid	ignored file ko force add karna
<code>cat ~/.gitignore_global</code>	basic	global gitignore file dekhna (sabhi repos ke liye)
<code>git config --global core.excludesfile ~/.gitignore_global</code>	mid	global gitignore set karna
<code>echo '*.log merge=union' >> .gitattributes</code>	adv	log files ke liye merge strategy set karna
<code>echo '*.jpg binary' >> .gitattributes</code>	adv	files ko binary mark karna (diff nahi dikhega)
<code>echo '*.sh eol=lf' >> .gitattributes</code>	adv	line endings force karna specific files ke liye
<code>git check-attr -a file.txt</code>	adv	file ke attributes dekhna

21. Git Internals — Andar Ki Baat

Command	Level	Kya karta hai
<code>git cat-file -t abc1234</code>	adv	object ka type dekhna (blob/tree/commit/tag)
<code>git cat-file -p abc1234</code>	adv	object ka content dekhna
<code>git ls-tree HEAD</code>	adv	current commit ka tree (files/dirs list)
<code>git ls-tree -r HEAD</code>	adv	recursive tree — sabhi files
<code>git ls-files</code>	adv	index (staging area) mein tracked files
<code>git ls-files -m</code>	adv	modified tracked files

Command	Level	Kya karta hai
<code>git ls-files -o --exclude-standard</code>	adv	untracked files dekhna
<code>git rev-parse HEAD</code>	adv	HEAD ka full SHA hash dekhna
<code>git rev-parse --short HEAD</code>	adv	short hash dekhna
<code>git rev-parse main</code>	adv	branch ka hash dekhna
<code>git rev-list --count main</code>	adv	branch pe total commits count
<code>git hash-object file.txt</code>	adv	file ka git hash calculate karna (store nahi karta)
<code>git symbolic-ref HEAD</code>	adv	current branch ka ref dekhna
<code>git for-each-ref --sort='-committerdate' refs/heads/</code>	adv	branches ko last commit date se sort karna
<code>git bundle create repo.bundle --all</code>	adv	entire repo ek file mein pack karna (offline transfer)
<code>git bundle verify repo.bundle</code>	adv	bundle valid hai ya nahi check karna
<code>git clone repo.bundle myrepo</code>	adv	bundle se repo restore karna

22. Commit Message Convention — Best Practice

Conventional Commits format: `type(scope): description` | Types: *feat, fix, docs, style, refactor, test, chore, perf, ci*

Type	Kab use karo	Example
feat	basic	naya feature add kiya — git log mein clearly dikhta hai
fix	basic	bug fix — release notes mein automatically aata hai
refactor	basic	code improve kiya, behavior same — test/feat nahi
docs	basic	sirf documentation change
test	basic	tests add/fix karna
chore	basic	build process, dependencies, config changes
perf	mid	performance improvement
ci	mid	CI/CD configuration changes
BREAKING CHANGE	adv	footer mein likho — major version bump trigger karta hai

23. Common Git Workflows

Command	Level	Kya karta hai
<code>git switch -c feature/ABC-123</code>	mid	Gitflow: naya feature branch banana (ticket number se)
<code>git push -u origin feature/ABC-123</code>	mid	Gitflow: branch remote pe push karna PR ke liye
<code>git fetch origin && git rebase origin/main</code>	mid	Gitflow: feature branch update karna latest main se

Command	Level	Kya karta hai
<code>git switch main && git merge --no-ff feature/ABC-123</code>	mid	Gitflow: feature merge with merge commit
<code>git tag -a v1.2.0 -m 'Release 1.2.0' && git push origin --tags</code>	mid	release tag banana aur push karna
<code>git switch -c hotfix/critical-bug main</code>	mid	Gitflow: production bug ke liye hotfix branch
<code>git log origin/main..HEAD --oneline</code>	mid	PR review: apne commits jo main mein nahi hain
<code>git diff origin/main...HEAD</code>	mid	PR review: apni branch ka complete diff
<code>git rebase -i origin/main</code>	adv	PR se pehle: commits squash/clean karna
<code>git push --force-with-lease</code>	adv	rebase ke baad safe force push karna

Golden Rules: (1) Shared/public branches pe kabhi rebase/force-push mat karo. (2) Commit often, push often — chote commits = easier review. (3) Kuch bhi karne se pehle git status dekho. (4) danger! commands se pehle git stash karo ya backup branch banana. (5) git reflog teri jaan bachayega agar kuch galat ho jaaye.